```
"""
SCANNER AVANSAT crypto + forex (nu execută ordine, doar dă semnale).
Doar Python standard, fără pip - merge în a-Shell pe iPhone.

Strategie (lumânări de 15 minute sau 1 oră, implicit 1 oră; doar lumânări ÎNCHISE
  LONG : EMA50 > EMA200 și preț peste trendul mare (≈ EMA50 pe 1 oră),
         închidere peste maximul ultimelor 20 lumânări,
         ADX >= 20 (piață cu trend), RSI între 50 și 75
  SHORT: invers (pe crypto doar pe Futures; pe Spot nu poți vinde fără monedă)
  Filtre: ținta trebuie să acopere comisioanele de 2 ori, forex doar în sesiunea
  Londra + New York (07:00-20:00 UTC), același semnal nu se repetă imediat pe ace
  Stop-loss 2 x ATR | Take-profit 3 x ATR | ieșire forțată după 30 lumânări
  Backtest realist: intrare la deschiderea lumânării următoare, comisioane inclus
  rezultate împărțite în 60% (antrenare) și 40% (test pe date nevăzute).

Rulare:
  python3 scanner.py all                          (implicit: lumânări de 1 oră)
  python3 scanner.py all --tf=15m                 (lumânări de 15 minute)
  python3 scanner.py crypto
  python3 scanner.py forex
  python3 scanner.py crypto BTCUSDT ETHUSDT       (doar perechile alese)
  python3 scanner.py all --cont=1000 --risc=1     (calculează și cantitatea)

Versiunea Telegram: adaugă --telegram ca să primești semnalele pe Telegram.
Cheile se citesc din variabilele TELEGRAM_TOKEN și TELEGRAM_CHAT_ID.
"""
import os
import sys
import json
import time
import urllib.parse
import urllib.request
from datetime import datetime, timezone
from concurrent.futures import ThreadPoolExecutor

CRYPTO_PAIRS = [
    "BTCUSDT", "ETHUSDT", "BNBUSDT", "SOLUSDT", "XRPUSDT", "ADAUSDT", "DOGEUSDT",
    "AVAXUSDT", "LINKUSDT", "DOTUSDT", "LTCUSDT", "TRXUSDT", "ATOMUSDT", "NEARUSD
    "UNIUSDT", "BCHUSDT", "XLMUSDT", "ETCUSDT", "APTUSDT", "ARBUSDT", "OPUSDT",
    "SUIUSDT", "INJUSDT", "FILUSDT", "TONUSDT", "HBARUSDT", "ICPUSDT", "AAVEUSDT"
    "RUNEUSDT", "ALGOUSDT", "VETUSDT", "SANDUSDT", "MANAUSDT", "GRTUSDT", "SEIUSD
    "TIAUSDT", "FETUSDT", "IMXUSDT", "STXUSDT", "LDOUSDT", "CRVUSDT", "EGLDUSDT",
    "THETAUSDT", "SHIBUSDT", "PEPEUSDT", "RENDERUSDT",
```

```
]
FOREX_PAIRS = [
    "EURUSD", "GBPUSD", "USDJPY", "USDCHF", "USDCAD", "AUDUSD", "NZDUSD",
    "EURGBP", "EURJPY", "EURCHF", "EURAUD", "EURCAD", "EURNZD",
    "GBPJPY", "GBPCHF", "GBPAUD", "GBPCAD", "GBPNZD",
    "AUDJPY", "AUDCAD", "AUDCHF", "AUDNZD",
    "NZDJPY", "NZDCAD", "NZDCHF", "CADJPY", "CADCHF", "CHFJPY",
]

DON, SL_ATR, TP_ATR, MAX_BARS, ADX_MIN = 20, 2.0, 3.0, 30, 20
COST_MULT = 2    # ținta (TP) trebuie să fie de cel puțin 2 x costul total al tra
CFG = {
    "crypto": {"fee": 0.001, "session": None},       # fee per intrare / per ieși
    "forex": {"fee": 0.0003, "session": (7, 20)},    # Londra + New York, ore UTC
}
# Setări pe interval: sec = secunde/lumânare, yrange = istoric Yahoo, pages = pag
# (x1000 lumânări), htf = EMA pentru trendul mare, age = vârsta maximă a semnalul
# cool = câte lumânări nu repetăm același semnal pe aceeași pereche.
TFS = {
    "5m": {"sec": 300, "yrange": "60d", "pages": 3, "htf": 600, "age": 15, "cool"
    "15m": {"sec": 900, "yrange": "60d", "pages": 5, "htf": 200, "age": 35, "cool
    "1h": {"sec": 3600, "yrange": "1y", "pages": 6, "htf": 200, "age": 75, "cool"
}
TF = BAR_SECONDS = YRANGE = PAGES = HTF_BARS = WARMUP = COOLDOWN = MAX_AGE = None
ONLY_QUALIFIED = True  # pe Telegram trimite doar perechile calificate


def set_tf(tf):
    global TF, BAR_SECONDS, YRANGE, PAGES, HTF_BARS, WARMUP, COOLDOWN, MAX_AGE
    t = TFS[tf]
    TF, BAR_SECONDS, YRANGE, PAGES = tf, t["sec"], t["yrange"], t["pages"]
    HTF_BARS, WARMUP, COOLDOWN, MAX_AGE = t["htf"], t["htf"] + 50, t["cool"], t["


set_tf("1h")


# ---------- date ----------
def http_json(url, tries=2):
    for k in range(tries):
        try:
            req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0
            with urllib.request.urlopen(req, timeout=25) as r:
                return json.loads(r.read().decode())
        except Exception:
            if k == tries - 1:
```

```
                raise
            time.sleep(1.5)


def fetch_crypto(sym, pages=None):
    pages = pages or PAGES
    out, end = [], None
    for _ in range(pages):
        url = f"https://data-api.binance.vision/api/v3/klines?symbol={sym}&interv
        if end:
            url += f"&endTime={end}"
        d = http_json(url)
        if not d:
            break
        out = [(int(k[0]) // 1000, float(k[1]), float(k[2]), float(k[3]), float(k
               for k in d] + out
        end = int(d[0][0]) - 1
    return out


def fetch_forex(pair):
    url = f"https://query1.finance.yahoo.com/v8/finance/chart/{pair}=X?interval={
    r = http_json(url)["chart"]["result"][0]
    q = r["indicators"]["quote"][0]
    hourly = []
    for t, o, h, l, c in zip(r["timestamp"], q["open"], q["high"], q["low"], q["c
        if None not in (o, h, l, c):
            hourly.append((t, o, h, l, c))
    # grupăm lumânările pe intervale de 5 minute (aliniate UTC)
    buckets, order = {}, []
    for t, o, h, l, c in hourly:
        k = t - t % BAR_SECONDS
        if k not in buckets:
            buckets[k] = [k, o, h, l, c]
            order.append(k)
        else:
            b = buckets[k]
            b[2], b[3], b[4] = max(b[2], h), min(b[3], l), c
    return [tuple(buckets[k]) for k in order]


# ---------- indicatori ----------
def wilder(vals, n):
    out = [vals[0]]
    for v in vals[1:]:
        out.append((out[-1] * (n - 1) + v) / n)
```

```
    return out


def ema(vals, n):
    k, out = 2 / (n + 1), [vals[0]]
    for v in vals[1:]:
        out.append(v * k + out[-1] * (1 - k))
    return out


def indicators(bars):
    closes = [b[4] for b in bars]
    gains, losses, pdm, mdm = [0.0], [0.0], [0.0], [0.0]
    tr = [bars[0][2] - bars[0][3]]
    for i in range(1, len(bars)):
        _, _, h, l, c = bars[i]
        ph, pl, pc = bars[i - 1][2], bars[i - 1][3], bars[i - 1][4]
        d = c - pc
        gains.append(max(d, 0.0))
        losses.append(max(-d, 0.0))
        up, dn = h - ph, pl - l
        pdm.append(up if up > dn and up > 0 else 0.0)
        mdm.append(dn if dn > up and dn > 0 else 0.0)
        tr.append(max(h - l, abs(h - pc), abs(l - pc)))
    atr = wilder(tr, 14)
    p, m = wilder(pdm, 14), wilder(mdm, 14)
    dx = []
    for i in range(len(bars)):
        if atr[i] == 0:
            dx.append(0.0)
            continue
        pdi, mdi = 100 * p[i] / atr[i], 100 * m[i] / atr[i]
        s = pdi + mdi
        dx.append(100 * abs(pdi - mdi) / s if s else 0.0)
    ag, al = wilder(gains, 14), wilder(losses, 14)
    rsi = [100.0 if al[i] == 0 else 100 - 100 / (1 + ag[i] / al[i]) for i in rang
    return {"e50": ema(closes, 50), "e200": ema(closes, 200), "ehtf": ema(closes,
            "adx": wilder(dx, 14), "rsi": rsi}


# ---------- strategie ----------
def signal(bars, ind, i, short_ok, cfg):
    if i < WARMUP or ind["adx"][i] < ADX_MIN:
        return None
    if cfg["session"]:
        hr = datetime.fromtimestamp(bars[i][0], timezone.utc).hour
```

```
        if not cfg["session"][0] <= hr < cfg["session"][1]:
            return None
    c, r = bars[i][4], ind["rsi"][i]
    # ținta trebuie să acopere comisioanele / spread-ul de cel puțin COST_MULT or
    if TP_ATR * ind["atr"][i] / c < COST_MULT * 2 * cfg["fee"]:
        return None
    hi = max(b[2] for b in bars[i - DON:i])
    lo = min(b[3] for b in bars[i - DON:i])
    if ind["e50"][i] > ind["e200"][i] and c > ind["ehtf"][i] and c > hi and 50 <=
        return "LONG"
    if (short_ok and ind["e50"][i] < ind["e200"][i] and c < ind["ehtf"][i]
            and c < lo and 25 <= r <= 50):
        return "SHORT"
    return None


def levels(side, entry, a):
    if side == "LONG":
        return entry - SL_ATR * a, entry + TP_ATR * a
    return entry + SL_ATR * a, entry - TP_ATR * a


def backtest(bars, ind, short_ok, cfg):
    trades, i = [], WARMUP
    while i < len(bars) - 2:
        side = signal(bars, ind, i, short_ok, cfg)
        if not side:
            i += 1
            continue
        entry = bars[i + 1][1]
        sl, tp = levels(side, entry, ind["atr"][i])
        last_j = min(i + MAX_BARS, len(bars) - 1)
        exit_p, j = None, i + 1
        while j <= last_j:
            h, l = bars[j][2], bars[j][3]
            if side == "LONG":
                if l <= sl:
                    exit_p = sl
                    break
                if h >= tp:
                    exit_p = tp
                    break
            else:
                if h >= sl:
                    exit_p = sl
                    break
```

```
                if l <= tp:
                    exit_p = tp
                    break
            j += 1
        if exit_p is None:
            j, exit_p = last_j, bars[last_j][4]
        ret = (exit_p - entry) / entry if side == "LONG" else (entry - exit_p) /
        trades.append((i, ret - 2 * cfg["fee"]))
        i = j + 1
    return trades


def stats(tr):
    if not tr:
        return {"n": 0, "pf": 0.0, "tot": 0.0, "wr": 0.0, "dd": 0.0}
    rets = [r for _, r in tr]
    w = sum(r for r in rets if r > 0)
    l = -sum(r for r in rets if r < 0)
    eq = peak = dd = 0.0
    for r in rets:
        eq += r
        peak = max(peak, eq)
        dd = max(dd, peak - eq)
    return {"n": len(rets), "pf": (w / l if l > 0 else (99.0 if w > 0 else 0.0)),
            "tot": sum(rets), "wr": sum(1 for r in rets if r > 0) / len(rets), "d


# ---------- analiză ----------
def analyze_pair(job):
    market, name = job
    try:
        bars = fetch_crypto(name) if market == "crypto" else fetch_forex(name)
        bars = bars[:-1]  # ignoră lumânarea încă neînchisă
        if len(bars) < WARMUP + 100:
            return {"name": name, "market": market, "err": "date insuficiente"}
        cfg = CFG[market]
        short_ok = True
        ind = indicators(bars)
        last = len(bars) - 1
        trades = backtest(bars, ind, short_ok, cfg)
        side = signal(bars, ind, last, short_ok, cfg)
        if side and any(signal(bars, ind, k, short_ok, cfg) == side
                        for k in range(last - COOLDOWN, last)):
            side = None  # semnal deja dat recent pe această pereche
        cut = int(len(bars) * 0.6)
        full = stats(trades)
```

```
        oos = stats([t for t in trades if t[0] >= cut])
        ok = (full["n"] >= 12 and full["pf"] >= 1.3 and full["tot"] > 0
              and oos["n"] >= 4 and oos["tot"] > 0)
        return {"name": name, "market": market, "err": None,
                "side": side,
                "price": bars[last][4], "atr": ind["atr"][last], "adx": ind["adx"
                "full": full, "oos": oos, "ok": ok, "close_t": bars[last][0] + BA
                "when": datetime.fromtimestamp(bars[last][0], timezone.utc).strft
    except Exception as ex:
        return {"name": name, "market": market, "err": str(ex)[:40]}


def size_line(r, sl, account, risk):
    dist = abs(r["price"] - sl)
    if not account or dist == 0:
        return None
    qty = account * risk / dist
    quote_usd = r["name"].endswith("USDT") or r["name"].endswith("USD")
    if quote_usd:
        return f"  Cantitate (risc {risk:.0%}): {qty:.6g} unități (~{qty * r['pri
    return "  Cantitate: calculeaz-o manual (moneda cotată nu e USD)"


def send_telegram(text):
    token = os.environ.get("TELEGRAM_TOKEN")
    chat = os.environ.get("TELEGRAM_CHAT_ID")
    if not token or not chat:
        print("Lipsesc TELEGRAM_TOKEN / TELEGRAM_CHAT_ID, nu trimit pe Telegram."
        return
    data = urllib.parse.urlencode({"chat_id": chat, "text": text}).encode()
    try:
        req = urllib.request.Request(f"https://api.telegram.org/bot{token}/sendMe
        urllib.request.urlopen(req, timeout=20)
    except Exception as ex:
        print("Eroare Telegram:", ex)


def signal_text(r, account=None, risk=0.01):
    sl, tp = levels(r["side"], r["price"], r["atr"])
    head = "🟢 CUMPĂRĂ" if r["side"] == "LONG" else "🔴 VINDE"
    tag = "✅ calificată" if r["ok"] else "⚠️ NECALIFICATĂ (risc mare)"
    txt = (f"{head} {r['name']} ({TF})\nIntrare ~{r['price']:.6g}\nStop-loss {sl:
           f"Take-profit {tp:.6g}\nBacktest: PF {r['full']['pf']:.2f}, {r['full']
           f"test nevăzut {r['oos']['tot']:+.1%}\n{tag}\n"
           f"Lumânare închisă la {datetime.fromtimestamp(r['close_t'], timezone.u
           "Nu intra dacă prețul s-a depărtat mult de intrare. Nu este sfat finan
```

```
    line = size_line(r, sl, account, risk)
    if r["market"] == "crypto" and r["side"] == "SHORT":
        txt += "\nAtenție: VINDE pe crypto = doar pe Futures (short). Pe Spot nu
    return txt + ("\n" + line.strip() if line else "")


def main():
    args = sys.argv[1:]
    telegram = "--telegram" in args
    if telegram:
        args.remove("--telegram")
    tf = "1h"
    for a in list(args):
        if a.startswith("--tf="):
            tf = a[5:]
            args.remove(a)
    if tf not in TFS:
        print("Interval necunoscut. Folosește --tf=15m sau --tf=1h")
        return
    set_tf(tf)
    account, risk = None, 0.01
    for a in list(args):
        if a.startswith("--cont="):
            account = float(a[7:])
            args.remove(a)
        elif a.startswith("--risc="):
            risk = float(a[7:]) / 100
            args.remove(a)
    if not args or args[0] not in ("crypto", "forex", "all"):
        print(__doc__)
        return
    market, custom = args[0], [x.upper() for x in args[1:]]
    jobs = []
    if market in ("crypto", "all"):
        jobs += [("crypto", p) for p in (custom if custom and market == "crypto"
    if market in ("forex", "all"):
        jobs += [("forex", p) for p in (custom if custom and market == "forex" el

    print(f"Analizez {len(jobs)} perechi pe lumânări de {TF} (poate dura 1-3 minu
    with ThreadPoolExecutor(max_workers=8) as ex:
        results = list(ex.map(analyze_pair, jobs))

    good = [r for r in results if not r["err"]]
    bad = [r["name"] for r in results if r["err"]]

    print("\n=== SEMNALE ACTIVE ===")
```

```
    active = sorted([r for r in good if r["side"]], key=lambda r: -r["adx"])
    if not active:
        print(f"Niciun semnal activ acum. Așteaptă următoarea lumânare de {TF}.")
    for r in active:
        sl, tp = levels(r["side"], r["price"], r["atr"])
        tag = "calificată" if r["ok"] else "NECALIFICATĂ (risc mare)"
        print(f"\n{'CUMPĂRĂ' if r['side'] == 'LONG' else 'VINDE'} {r['name']} | {
        print(f"  Intrare ~{r['price']:.6g} | SL {sl:.6g} | TP {tp:.6g}")
        print(f"  Backtest: PF {r['full']['pf']:.2f} | {r['full']['n']} tranz. |
              f"test nevăzut {r['oos']['tot']:+.1%}")
        line = size_line(r, sl, account, risk)
        if line:
            print(line)

    print("\n=== PERECHI CU BACKTEST BUN ===")
    quals = sorted([r for r in good if r["ok"]], key=lambda r: -r["oos"]["tot"])[
    if not quals:
        print("Nicio pereche nu trece filtrul. Mai bine nu tranzacționa acum.")
    for r in quals:
        print(f"{r['name']:<9} PF {r['full']['pf']:.2f} | {r['full']['n']} tranz.
              f"total {r['full']['tot']:+.1%} | test nevăzut {r['oos']['tot']:+.1

    print(f"\nScanate {len(good)} | calificate {sum(1 for r in good if r['ok'])}
    if bad:
        print("Fără date: " + ", ".join(bad[:10]) + ("..." if len(bad) > 10 else
    print("\nAtenție: nu este sfat financiar. Un backtest bun nu garantează profi
    print("iar alegerea perechilor cu cel mai bun istoric poate fi doar noroc.")

    if telegram:
        now = datetime.now(timezone.utc)
        manual = os.environ.get("GITHUB_EVENT_NAME") == "workflow_dispatch"
        for r in active:
            fresh = now.timestamp() - r["close_t"] <= MAX_AGE
            if manual or (fresh and (r["ok"] or not ONLY_QUALIFIED)):
                send_telegram(signal_text(r, account, risk))
        early = now.minute < BAR_SECONDS // 60
        if len(bad) > len(results) / 2 and (manual or (early and now.hour % 8 ==
            send_telegram(f"⚠️ Scanner: {len(bad)} din {len(results)} perechi făr
                          "Sursa de date poate fi blocată sau limitată.")
        elif manual or (early and now.hour == 8 and not active):
            send_telegram(f"✅ Scanner {TF} activ. Scanate {len(good)}, calificat
                          f"{sum(1 for r in good if r['ok'])}, semnale acum: {len


if __name__ == "__main__":
    main()
```